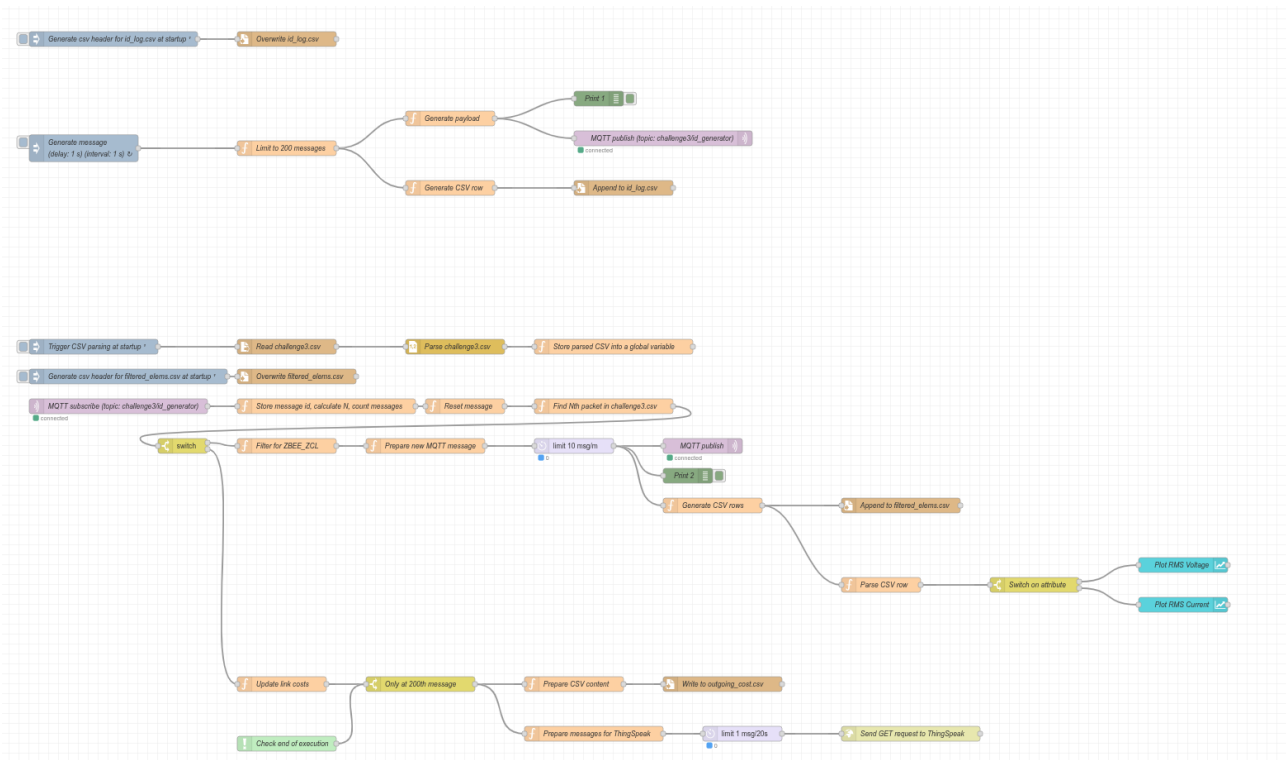


IOT 2025/2026 - Challenge 3

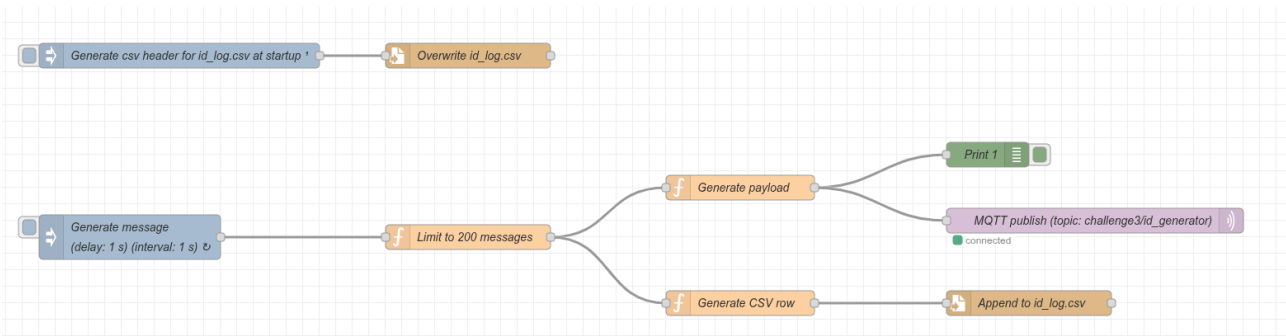
PART 1: NodeRed

Simone Amighini (person code: 10832147)

Overall picture of the flow



Section 1: message generation and logging to *id_log.csv*



- Generate csv header... (Inject) & Overwrite id_log.csv (File): These work together once at startup to wipe the old log and write the column headers.
- Generate message (Inject) – Fires every 1 second and creates a message containing a random ID between 0 and 3000 and the UNIX timestamp.
- Limit to 200 messages (Function) – It increments a local counter and stops the flow once 200 messages have passed. **Even if not explicitly requested, it should be done to stop the**

generation of the messages and also because after the 200th message, no data is going to be read or used in other parts of the flow (see stop condition in *Section 3*).

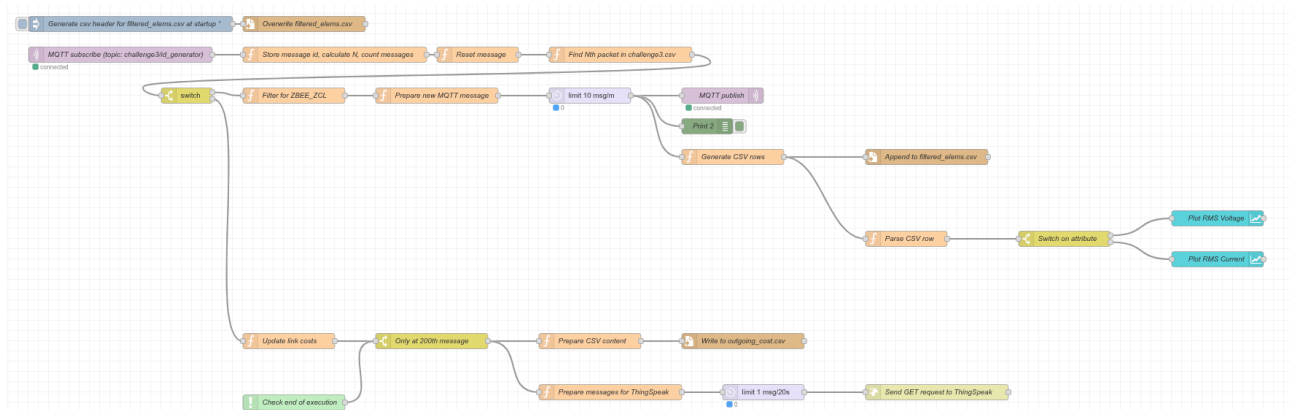
- Generate payload (Function) – A "string builder" that converts the ID and timestamp into a JSON-formatted string which is written into *msg.payload*.
- Print 1 (Debug) – Prints that JSON string to the debug sidebar.
- MQTT publish (MQTT Out) – Sends the generated ID to the *challenge3/id_generator* topic on the local broker on port 1884.
- Generate CSV row (Function) – Prepares a line of text (Counter, ID, Timestamp) for the following node.
- Append to id_log.csv (File) – Append that line to the file.

Section 2: *challenge3.csv* reading



- Trigger CSV parsing (Inject) & Read challenge3.csv (File In) & Parse challenge3.csv (CSV) & Store parsed CSV... (Function) – Read *challenge3.csv*, convert raw text into a structured JavaScript Array of objects and stores it into global memory so that any other node can access it later.

Section 3: process generated messages



- MQTT subscribe (MQTT In) – Reads all the messages generated in section 1 (topic *challenge3/id_generator* from the local broker).
- Store message id... (Function) – Calculates the index N using the formula $N = ID \bmod 5218$ and tracks how many messages have been received.
- Reset message (Function) – Clears out useless data from *msg* object.
- Find Nth packet... (Function) – Reaches into the global memory from Section 2 and pulls out the specific row of *challenge3.csv* corresponding to index N .
- Switch – Checks the *Packet_Type*. If it's a "Link Status" packet, it goes to one path; everything else goes to the main processing path.

- Main processing path:
 - Filter for ZBEE_ZCL (Function) – A secondary check to ensure we only care about ZigBee Cluster Library commands.
 - Prepare new MQTT message (Function) – It generates the *msg.topic* and *msg.payload* (containing *CURRENT_TIMESTAMP*, *SUB_ID*, *SRC_ADDR*, *DST_ADDR*, *ZB_SRC_ADDR*, *ZB_DST_ADDR*, *ZB_SRC_DEV_NAME* and *CMD_STR* as specified by the challenge) for messages that are going to be sent via MQTT.
 - Delay (Delay) – Limits to 10 messages per minute the rate of transmission to the local MQTT broker.
 - Generate CSV rows (Function) & Append to *filtered_elems.csv* (File) – Look for specific power metrics (RMS Voltage, RMS Current, Active Power) inside the command string, format them for logging and write them into *filtered_elems.csv*. **If a message doesn't contain the "Status" array following the "Attributes", it is discarded while if only some values are missing they are replaced by the default value 0.**
 - Parse CSV row (Function) & Switch on attribute (Switch) & Plot RMS Voltage/Current (UI-Chart) – Convert the CSV string back to JSON messages and route them to the correct chart in order to be plot. Dashboards are visible on <http://localhost:1880/dashboard/page1>.
- "Link status" packets path:
 - Update link costs (Function) – If the packet was "Link Status," this node updates a map containing source, destination and cost for each link documented in incoming packets.
 - Check end of execution (Complete) – Monitors the processing nodes. When they finish their work on the 200th message, this triggers.
 - Prepare CSV content (Function) & Write to *outgoing_cost.csv* (File) – Once the end of execution is triggered, it summarizes the "Link Cost" map and saves it as a final report into *outgoing_cost.csv*.
 - Prepare messages for ThingSpeak (Function) – Finds the device with the lowest ID, sorts its connections, and builds GET requests to transmit data to ThingSpeak.
 - Delay (Delay) – It ensures messages are sent 20 seconds apart to avoid blocks from ThingSpeak.
 - Send GET request (HTTP Request) – Pushes the GET request prepared before to my ThingSpeak public channel (<https://thingspeak.mathworks.com/channels/3377806>).

Obtained NodeRed charts

